B. Variety is Discouraged

time limit per test 1.5 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1100 | Tags: binary search, constructive algorithms, greedy, two pointers |
https://codeforces.com/problemset/problem/2064/B

Define the score of an arbitrary array b to be the length of b minus the number of distinct elements in b. For example:

- The score of [1, 2, 2, 4] is 1, as it has length 4 and only 3 distinct elements (1, 2, 4).
- The score of [1, 1, 1] is 2, as it has length 3 and only 1 distinct element (1).
- The empty array has a score of 0.

You have an array a. You need to remove some **non-empty** contiguous subarray from a **at most** once.

More formally, you can do the following **at most** once:

- pick two integers l, r where $1 \le l \le r \le n$, and
- delete the contiguous subarray $[a_l,\ldots,a_r]$ from a (that is, replace a with $[a_1,\ldots,a_{l-1},a_{r+1},\ldots,a_n]$).

Output an operation such that the score of a is **maximum**; if there are multiple answers, output one that **minimises** the final length of a after the operation. If there are still multiple answers, you may output any of them.

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of testcases.

The first line of each testcase contains an integer n ($1 \le n \le 2 \cdot 10^5$) — the length of the array a.

The second line of each testcase contains n integers $a_1,a_2,\ldots,a_n$ ($1 \le a_i \le n$).

The sum of n across all testcases does not exceed $2 \cdot 10^5$.

Output

For each testcase, if you wish to not make a move, output 0.

Otherwise, output two integers l and r ($1 \le l \le r \le n$), representing the left and right bound of the removed subarray.

The removed subarray should be chosen such that the score is maximized, and over all such answers choose any of them that minimises the final length of the array.

Example

Input
3 1 1 5 1 1 1 1 1 4 2 1 3 2

Output
1 1 0 2 3

Note

In the first testcase, we have two options:

- do nothing: the score of [1] is 1-1=0.
- remove the subarray with *l*=1, *r*=1: we remove the only element, and we get an empty array with score 0.

Therefore, the maximum score possible is 0. However, since we need to additionally minimise the final length of the array, we **must** output the second option with *l*=*r*=1. Note that the first option of doing nothing is **incorrect**, since it has a longer final length.

In the second testcase, no subarray is selected, so after which *a* is still [1, 1, 1, 1, 1]. This has length 5 and 1 distinct element, so it has a score of 5 - 1 = 4. This can be proven to be a shortest array which maximises the score.

In the third testcase, the subarray selected is [2, color*red*1, color*red*3, 2], after which *a* becomes [2, 2]. This has length 2 and 1 distinct element, so it has a score of 2 - 1 = 1. This can be proven to be a shortest array which maximises the score.